Search Write
Get unlimited access to the best of Medium for less than $1/week. Become a member

Intermediate SQL (Part 1)

Faisal Afif · Follow
Published in Data Science Indonesia · 9 min read · Nov 21, 2023

24

> *A beginner guide for learning SQL from scratch*

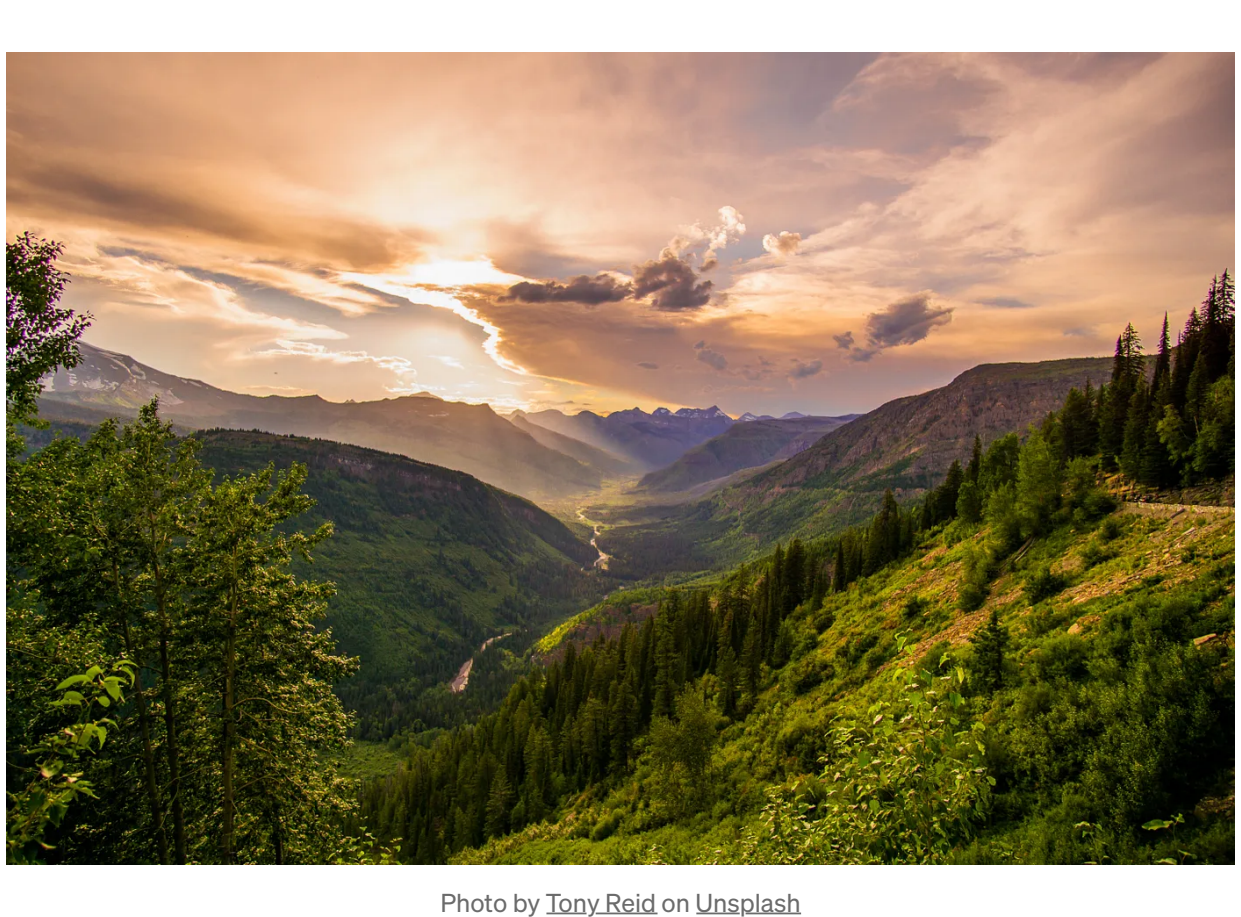


Photo by Tony Reid on Unsplash

So far we have already learned some basic SQL queries, such as SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, and LIMIT. If you would like to review them, please visit this Introduction to SQL series.

Introduction to SQL (Part 1)
A beginner guide for learning SQL from scratch
medium.com

The intermediate SQL series will cover the following topics:

- Combining multiple tables using JOINs and set operations
- Performing window functions in SQL
- Differentiating between subqueries, common table expressions (CTEs), views and temporary tables
- Modifying rows and tables

* * *

In this section, we will explore JOINs and set operations. JOINs are essential for combining data from multiple tables to answer complex questions. We will start with the basic JOINs and then progress to advanced JOINs techniques, and last but not least we will explore set operations in SQL.

* * *

Before we dive into the different types of JOINs and how to use them, let's review the principles. In SQL, a **JOIN is an operation that combines rows from two or more tables based on a related column between them. This relationship is often established through primary keys and foreign keys.**

A primary key is a field in a table that uniquely identifies each record in that table. On the other hand, a foreign key is a field in a table that references the primary key of another table. This relationship is a fundamental aspect of a relational database.

To illustrate, consider these two tables: `Movies` and `BoxOffice`. In the `Movies` table, the `Id` field serves as the primary key, uniquely identifying each movie record. In the `BoxOffice` table, the `Movie_id` field acts as a foreign key, referencing the `Id` field in the `Movies` table. This relationship enables the JOIN operation to connect relevant information from both tables.

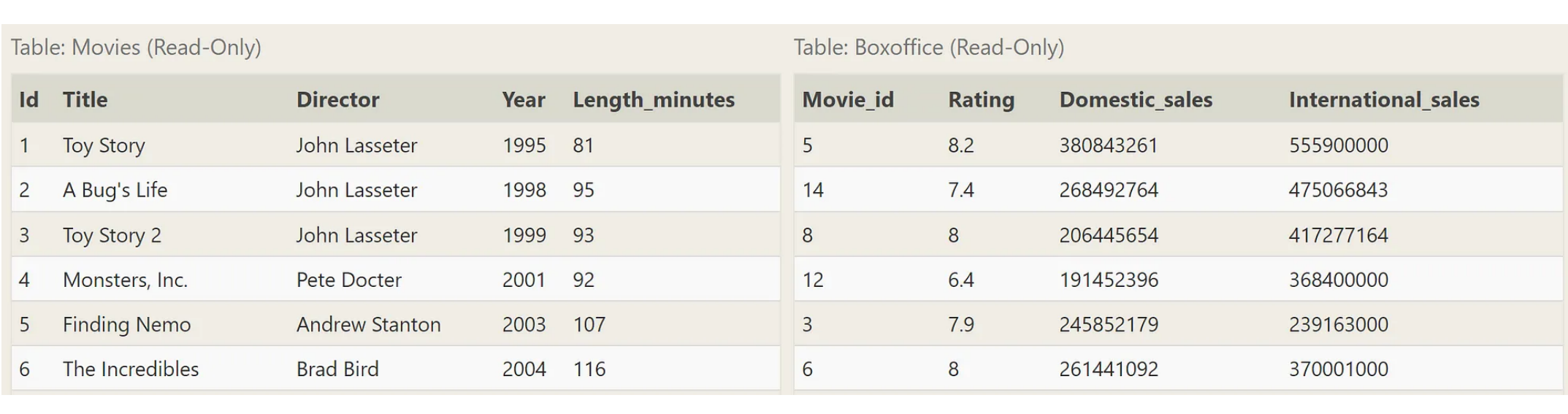


Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116

Table: Boxoffice (Read-Only)

Movie_id	Rating	Domestic_sales	International_sales
5	8.2	380843261	555900000
14	7.4	268492764	475066843
8	8	206445654	417277164
12	6.4	191452396	368400000
3	7.9	245852179	239163000
6	8	261441092	370001000

Figure 1. Movies Table and Box Office Table (from SQL Bolt)

SQL offers several types of JOINs, each with a unique purpose. These are several types of JOINs.

1. INNER JOIN

An **`INNER JOIN`** between two tables returns only records where the **`JOIN`** condition matches in both tables. To illustrate, consider the `Movies` and `BoxOffice` tables from the SQL Bolt. When performing an **`INNER JOIN`**, the resulting data will include only those records where the `id` in the `Movies` table matches the `movie_id` in the `BoxOffice` table.

```
-- Inner Join between Movies table and Box Office table
-- Primary key in Movies table = id
-- Foreign key in BoxOffice table = movie_id
SELECT
    *
FROM
    movies AS m
INNER JOIN
    boxoffice AS bo
    ON m.id=bo.movie_id;
```

2. LEFT JOIN and RIGHT JOIN

A **`LEFT JOIN`** keeps all the original records from the left table and includes missing values (**`NULL`**) for any columns from the right table where the joining field did not find a match. On the other hand, a **`RIGHT JOIN`** keeps all the original records from the right table and includes missing values (**`NULL`**) for any columns from the left table where the joining field did not find a match. **`RIGHT JOIN`** is far less common than **`LEFT JOIN`**, because **`RIGHT JOIN`** can always be rewritten as **`LEFT JOIN`**.

For example, `Buildings` and `Employess` tables below from the SQL Bolt. When performing a **`LEFT JOIN`**, the resulting data will include all the records from `Buildings` table and includes missing values (**`NULL`**) when `Building_name` column did not have any match with `Building` column from `Employees` table.

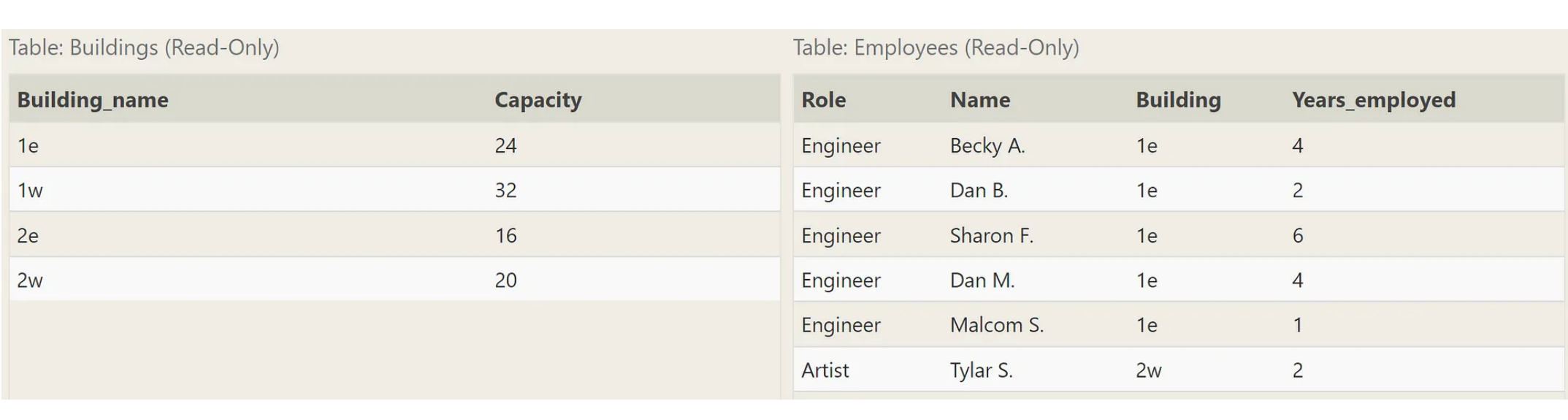


Table: Buildings (Read-Only)

Building_name	Capacity
1e	24
1w	32
2e	16
2w	20

Table: Employees (Read-Only)

Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2

Figure 2. Buildings Table and Employees Table

```
-- Left Join between Building table and Employees table
-- Primary key in Building table = building_name
-- Foreign key in Employees table = building
SELECT
    *
FROM
    buildings AS b
LEFT JOIN
    employees AS e
    ON b.building_name=e.building;

-- Right Join between Building table and Employees table (returns all records fr
-- Primary key in Building table = building_name
-- Foreign key in Employees table = building
SELECT
    *
FROM
    buildings AS b
RIGHT JOIN
    employees AS e
    ON b.building_name=e.building;
```

3. FULL JOIN

A **`FULL JOIN`** is a combination of **`LEFT JOIN`** and **`RIGHT JOIN`**. It returns all records from both tables involved, regardless of whether there is a match on the joining field in the other table. In cases where a match is not found, the result will include `NULL` values for the columns from the table without a matching record.

```
-- Full Join between Building table and Employees table
-- Primary key in Building table = building_name
-- Foreign key in Employees table = building
SELECT
    *
FROM
    buildings AS b
FULL OUTER JOIN
    employees As e
    ON b.building_name=e.building;
```

4. SELF JOIN

The **`SELF JOIN`**, as its name implies, is used to join a table to itself. This means that each row in a table is joined to itself and every other row in that table. However, referencing the same table more than once within a single query will result in an error. To avoid this, in **`SELF JOIN`** aliases are used. Either **`LEFT JOIN`** or **`INNER JOIN`** is commonly used to perform a **`SELF JOIN`**

```
-- Syntax for Self Join
SELECT
    column_names
FROM
    table_name AS t1
[INNER | LEFT]  JOIN table_name AS t2
    ON join_predicate;


-- Self Join of Movies table where the same Director has created two movies and

SELECT
    m1.title AS title_1,
    m2.title AS title_2,
    m1.year AS year_1,
    m2.year AS year_2,
    m1.director
FROM
    movies AS m1
INNER JOIN
    movies AS m2
    ON m1.director=m2.director
    AND m1.year<m2.year;
```

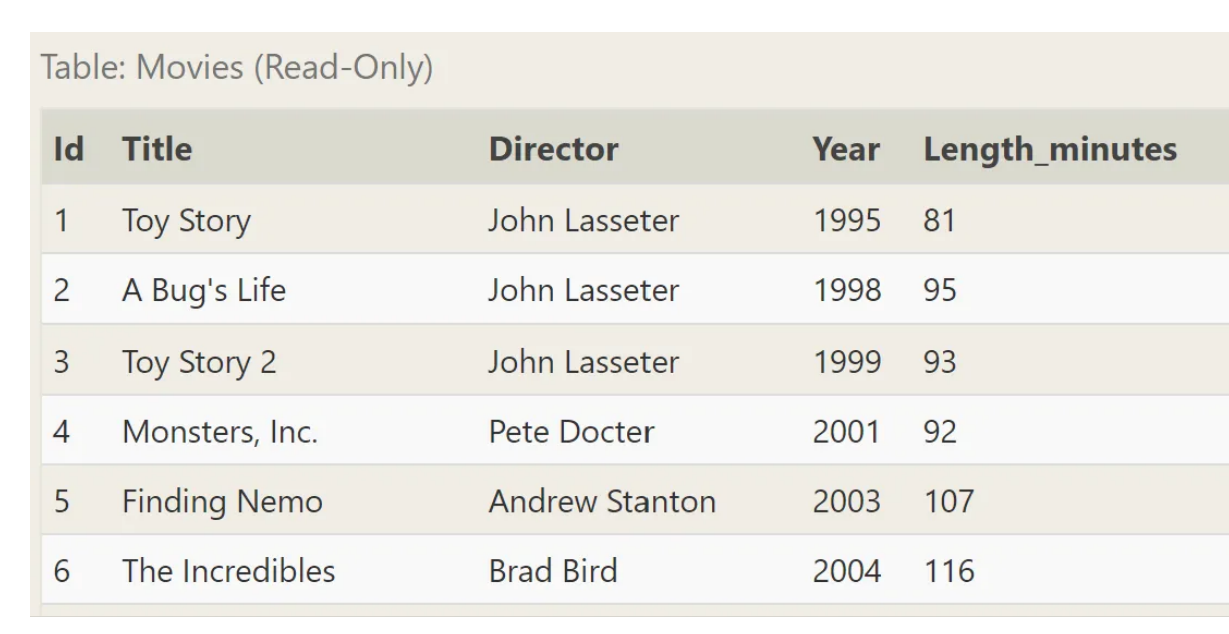
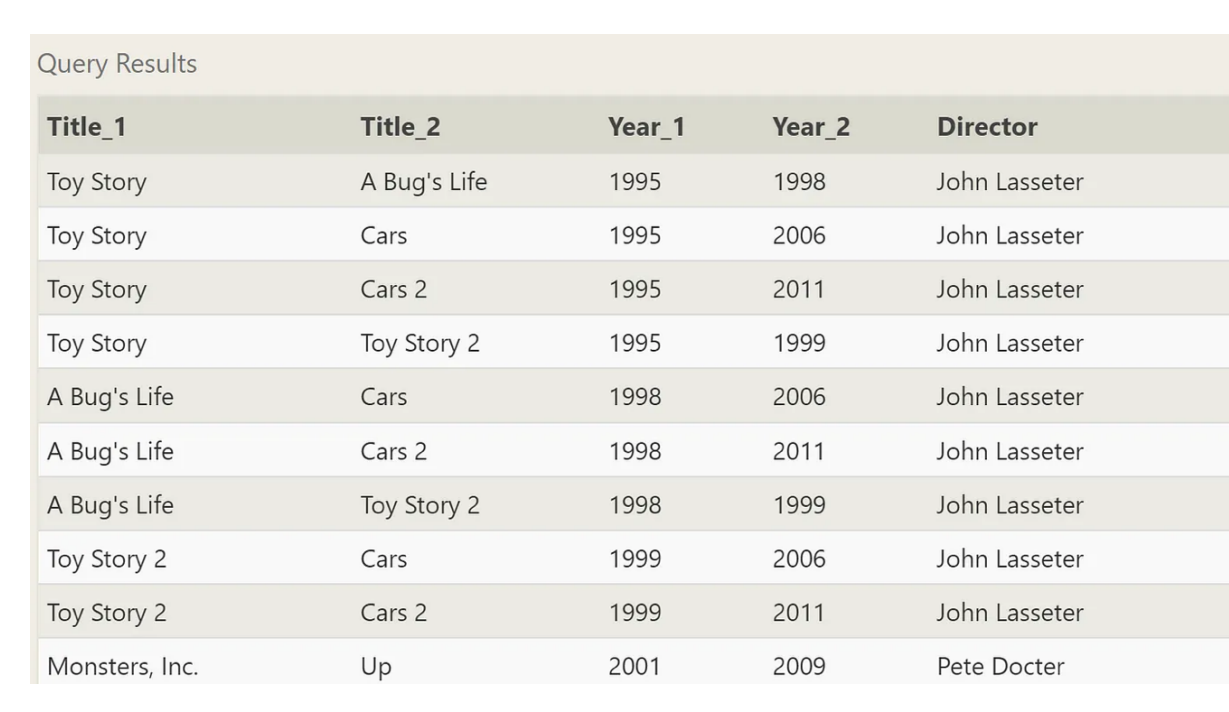


Table: Movies (Read-Only)

Id	Title	Director	Year	Length_minutes
1	Toy Story	John Lasseter	1995	81
2	A Bug's Life	John Lasseter	1998	95
3	Toy Story 2	John Lasseter	1999	93
4	Monsters, Inc.	Pete Docter	2001	92
5	Finding Nemo	Andrew Stanton	2003	107
6	The Incredibles	Brad Bird	2004	116

Figure 3. Movies Table (from SQL Bolt)



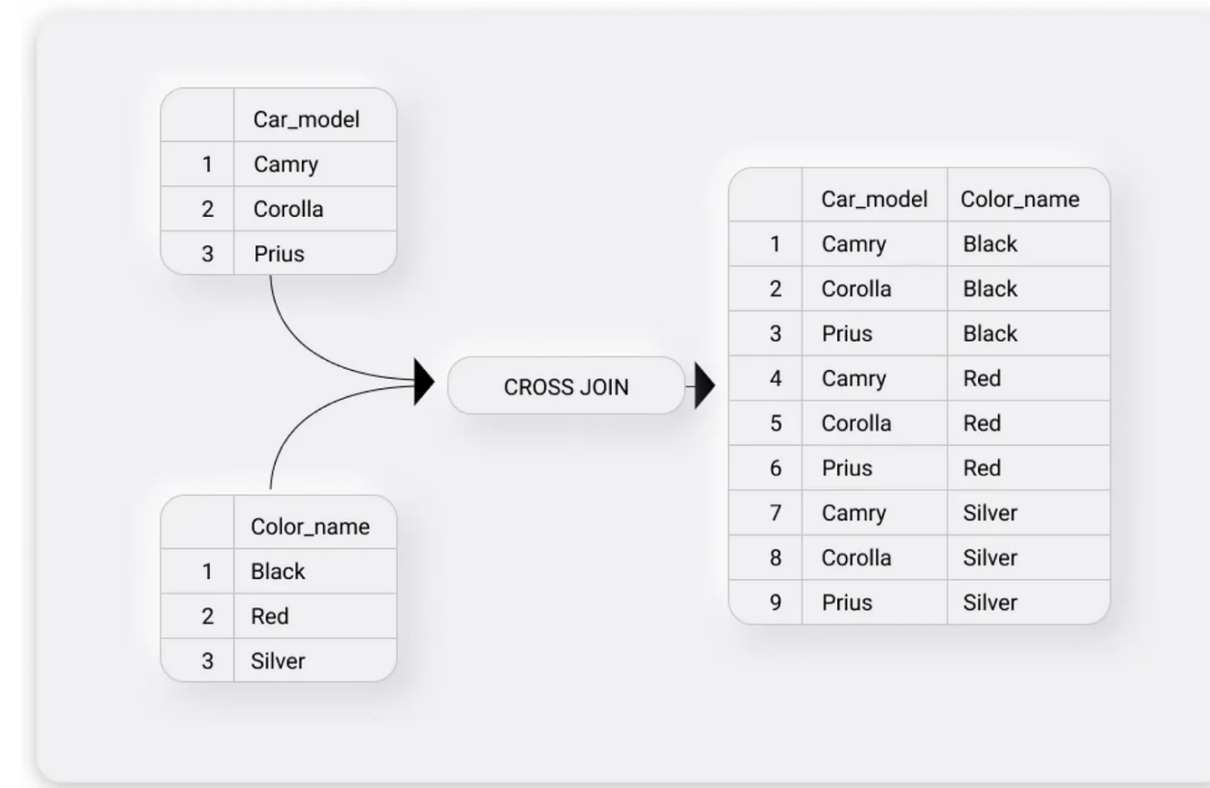
Query Results

Title_1	Title_2	Year_1	Year_2	Director
Toy Story	A Bug's Life	1995	1998	John Lasseter
Toy Story	Cars	1995	2006	John Lasseter
Toy Story	Cars 2	1995	2011	John Lasseter
Toy Story	Toy Story 2	1995	1999	John Lasseter
A Bug's Life	Cars	1998	2006	John Lasseter
A Bug's Life	Cars 2	1998	2011	John Lasseter
A Bug's Life	Toy Story 2	1998	1999	John Lasseter
Toy Story 2	Cars	1999	2006	John Lasseter
Toy Story 2	Cars 2	1999	2011	John Lasseter
Monsters, Inc.	Up	2001	2009	Pete Docter

Figure 4. Result of Self Join (Example by using Movies table)

5. CROSS JOIN

The **`CROSS JOIN`** in SQL is commonly used to generate all combinations of records in two tables. For example, you have two columns: `car_model` and `color_name`, and you need a result set to display all the possible paired combinations of those (cartesian product).



	Car_model
1	Camry
2	Corolla
3	Prius

	Color_name
1	Black
2	Red
3	Silver

	Car_model	Color_name
1	Camry	Black
2	Corolla	Black
3	Prius	Black
4	Camry	Red
5	Corolla	Red
6	Prius	Red
7	Camry	Silver
8	Corolla	Silver
9	Prius	Silver

Figure 5. Combining two tables with Cross Join

```
-- Syntax for Cross Join
SELECT
    column_1, column_2
FROM
    table_1
CROSS JOIN
    table_2;

-- Cross Join of car_model from Cars table and color_name from Colors table
SELECT
    c.Car_model,
    c1.Color_name
FROM
    Cars AS c
CROSS JOIN
    Colors AS c1;
```

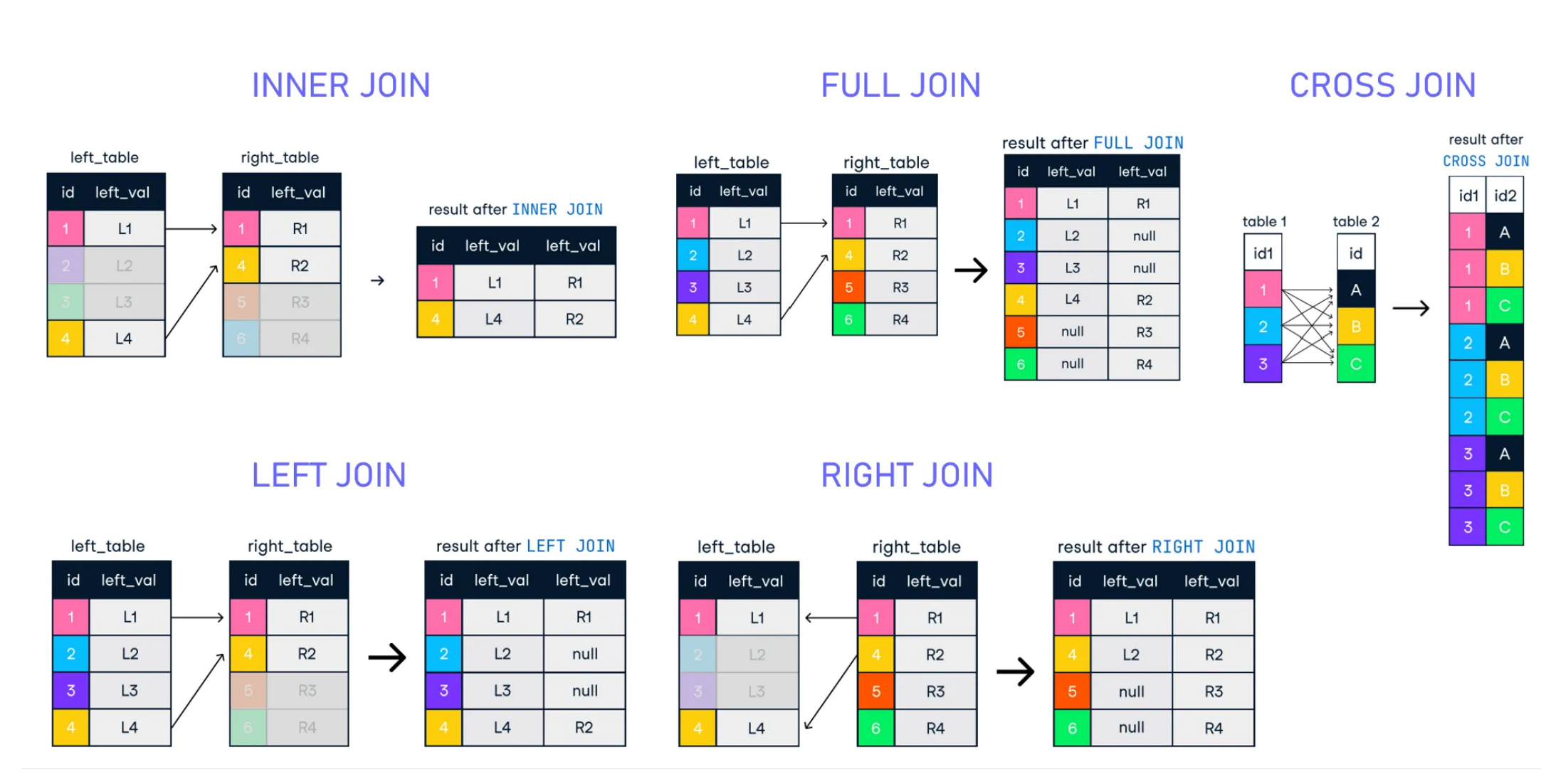


Figure 6. Summary of JOINS from DataCamp SQL JOINS Cheat Sheet

6. SET OPERATIONS

Set operations in SQL is a type of operations which allow the results of multiple queries to be combined into a single result set. Set operators in SQL include **`UNION`**, **`UNION ALL`**, **`INTERSECT`**, and **`EXCEPT`**, which correspond to the concepts of union, intersection and set difference in mathematics.

To combine two or more queries into a single result set, they must follow these requirements:

- The result sets of both queries must have the same number of columns.
- The corresponding columns in the two queries must have the same data type or must be implicitly convertible to the same data type.

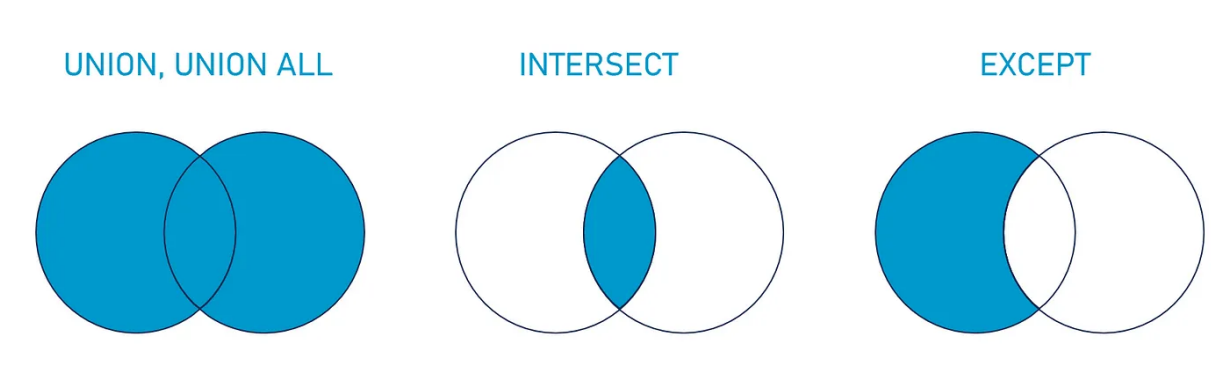


Figure 7. Illustration of set operations (Image by author)

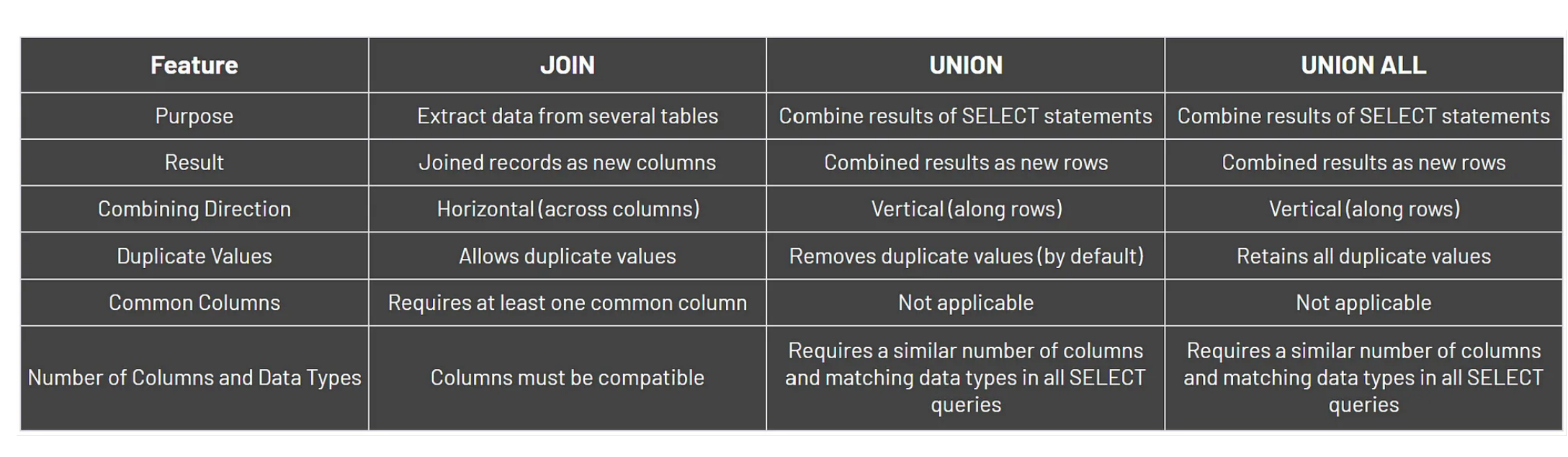
6.1. UNION and UNION ALL

When working with multiple tables, **`UNION`** and **`UNION ALL`** operators allow you to combine the results of two or more **`SELECT`** statements into a single result set. The **`UNION`** operator removes duplicate rows between the tables by default, while the **`UNION ALL`** operator does not.

```
-- Syntax for UNION
SELECT
    column_1, column_2
FROM
    table_1
UNION
SELECT
    column_1, column_2
FROM
    table2;

-- Syntax for UNION ALL
SELECT
    column_1, column_2
FROM
    table_1
UNION ALL
SELECT
    column_1, column_2
FROM
    table2;
```

Here is a comparison of JOIN, UNION and UNION ALL to get clear understanding of their differences.



Feature	JOIN	UNION	UNION ALL
Purpose	Extract data from several tables	Combine results of SELECT statements	Combine results of SELECT statements
Result	Joined records as new columns	Combined results as new rows	Combined results as new rows
Combining Direction	Horizontal (across columns)	Vertical (along rows)	Vertical (along rows)
Duplicate Values	Allows duplicate values	Removes duplicate values (by default)	Retains all duplicate values
Common Columns	Requires at least one common column	Not applicable	Not applicable
Number of Columns and Data Types	Columns must be compatible	Requires a similar number of columns and matching data types in all SELECT queries	Requires a similar number of columns and matching data types in all SELECT queries

Figure 8. Difference between JOIN, UNION and UNION ALL (Image by author)

6.2. INTERSECT

The **`INTERSECT`** operator takes the results of two queries and returns only rows that are identical in both result sets. The **`INTERSECT`** operator removes duplicate rows from the final result set. Whereas a **`UNION`** operator is a logical **`OR`**, **`INTERSECT`** is a logical **`AND`**.

```
-- Syntax for INTERSECT
SELECT
    column_1, column_2
FROM
    table_1
INTERSECT
SELECT
    column_1, column_2
FROM
    table2;
```

6.3. EXCEPT

The **`EXCEPT`** operator returns only those rows from the first result set that are not in the second result set. This means that the **`EXCEPT`** operator is query order-sensitive, like the **`LEFT JOIN`** and **`RIGHT JOIN`**. In some SQL flavors, they provide **`MINUS`** operator which is functionally equivalent to the **`EXCEPT`**.

```
-- Syntax for EXCEPT
SELECT
    column_1, column_2
FROM
    table_1
EXCEPT
SELECT
    column_1, column_2
FROM
    table2;
```

When combining multiple queries by using **`UNION`**, **`UNION ALL`**, **`INTERSECT`** and **`EXCEPT`**, you can add another clause like **`ORDER BY`** and **`LIMIT`**.

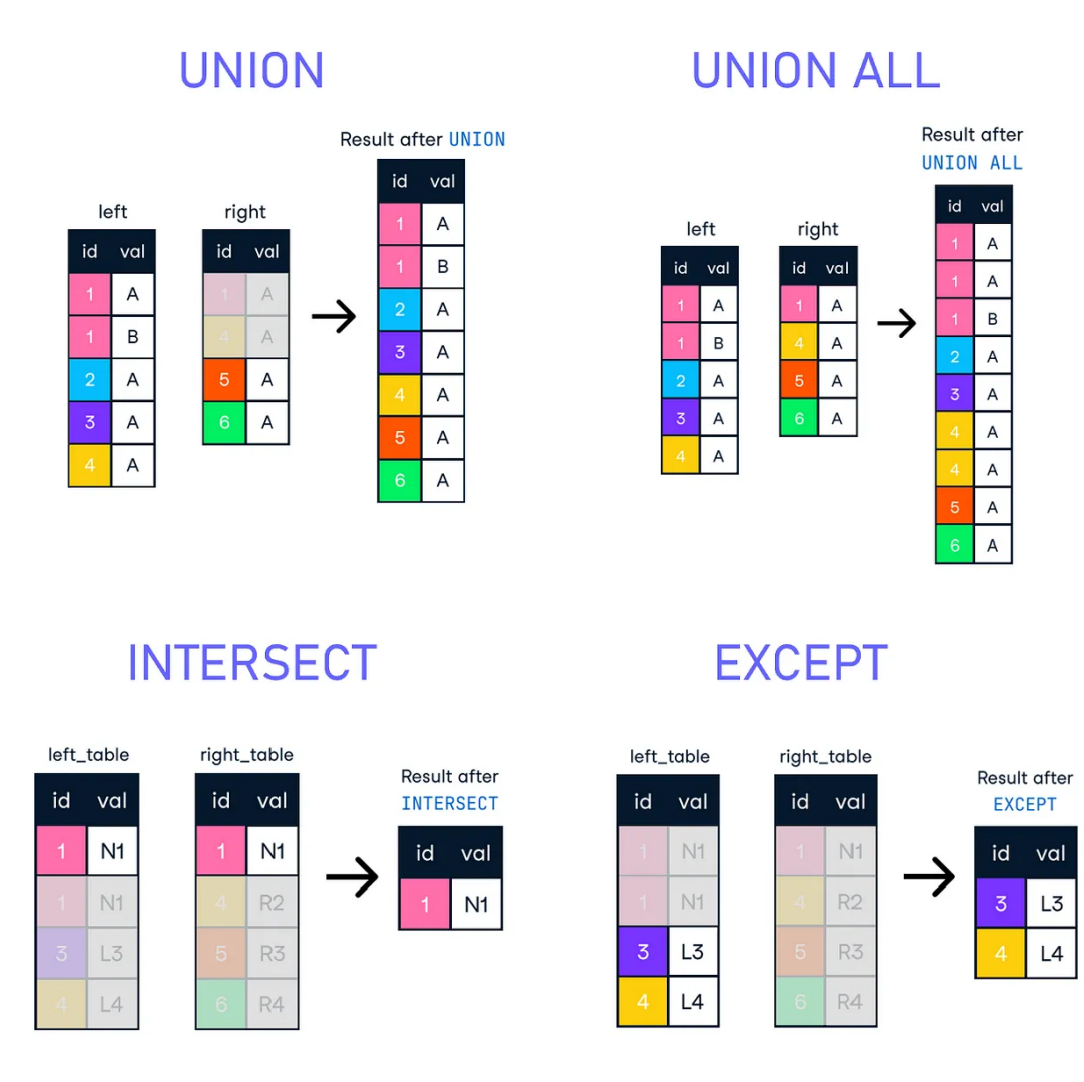


Figure 9. Summary of SET OPERATIONS from DataCamp SQL JOINS Cheat Sheet

7. SEMI JOIN and ANTI JOIN

A **`SEMI JOIN`** returns rows from the first table where a condition is met in the second table, but it does not include the actual columns from the second table in the result set. It uses a **`WHERE`** clause and a simple subquery to filter the first table based on a specific condition. Whereas an **`ANTI JOIN`** returns rows from the first table where a condition is **`NOT`** met in the second table. It also uses a **`WHERE`** clause and simple subquery.

For example, we want to get `Building_name` and `Capacity` from `Buildings` table where

1. `Building` is found in the `Employees` table
2. `Building` is not found in the `Employees` table

```
-- To get data in the Buildings table where Building is found in the Employee ta
SELECT
    *
FROM
    Buildings
WHERE
    Building_name IN
(SELECT
    Building
FROM
    Employees)
;

-- Data in the Buildings table where Building is not found in the Employee table
SELECT
    *
FROM
    Buildings
WHERE
    Building_name NOT IN
(
SELECT
    Building
FROM
    Employees);
```

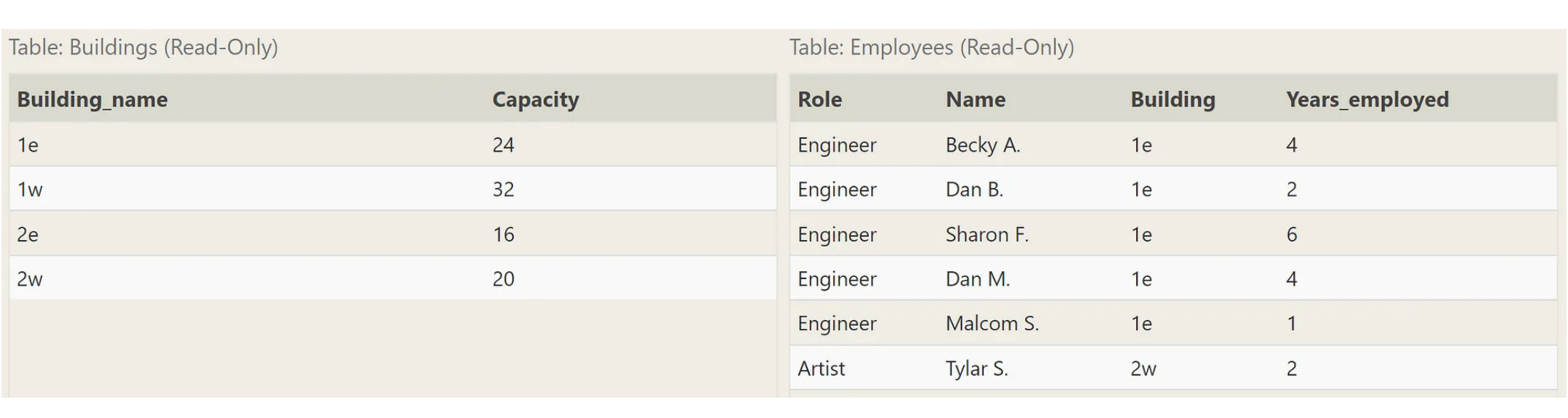


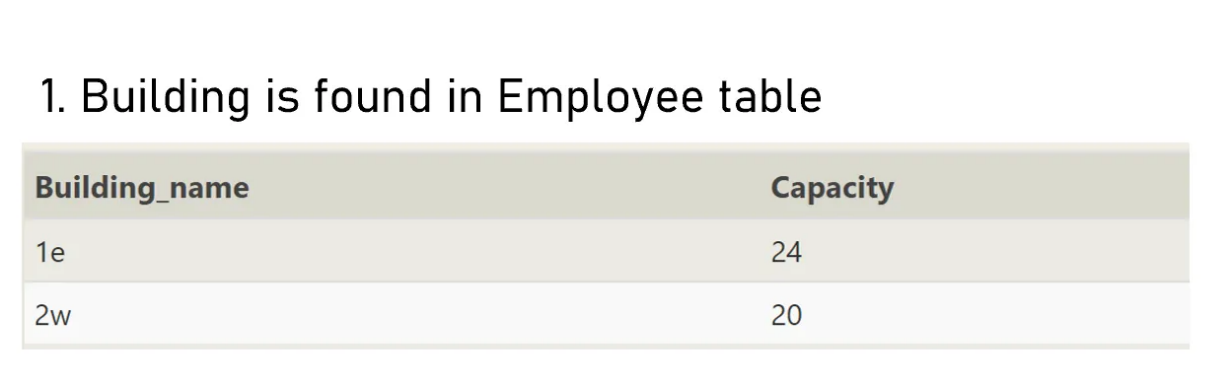
Table: Buildings (Read-Only)

Building_name	Capacity
1e	24
1w	32
2e	16
2w	20

Table: Employees (Read-Only)

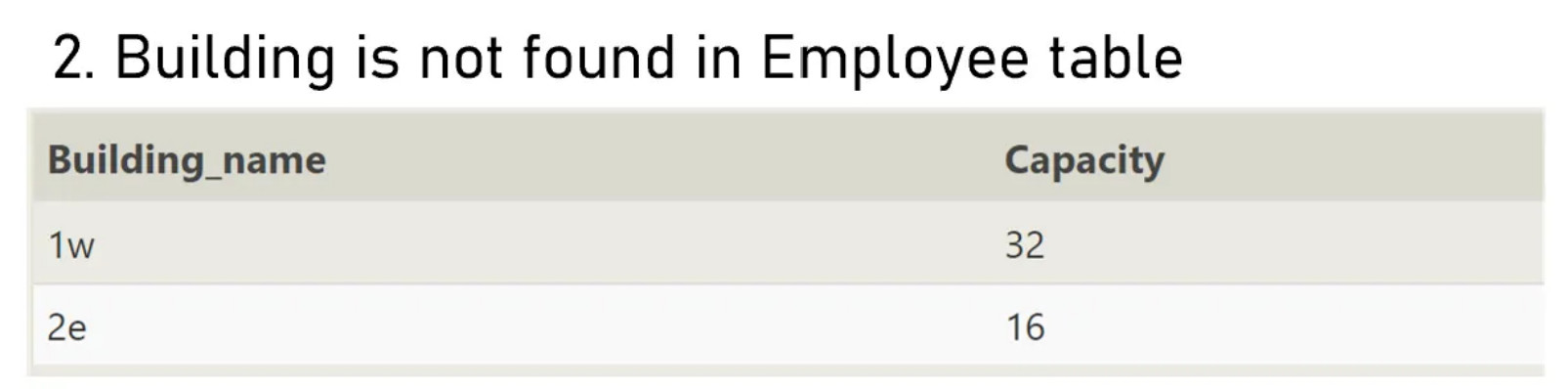
Role	Name	Building	Years_employed
Engineer	Becky A.	1e	4
Engineer	Dan B.	1e	2
Engineer	Sharon F.	1e	6
Engineer	Dan M.	1e	4
Engineer	Malcom S.	1e	1
Artist	Tylar S.	2w	2

Figure 10. Buildings Table and Employees Table



1. Building is found in Employee table

Building_name	Capacity
1e	24
2w	20



2. Building is not found in Employee table

Building_name	Capacity
1w	32
2e	16

Figure 11. Result of SEMI JOIN and ANTI JOIN (Example by using Buildings Table and Employees Table)

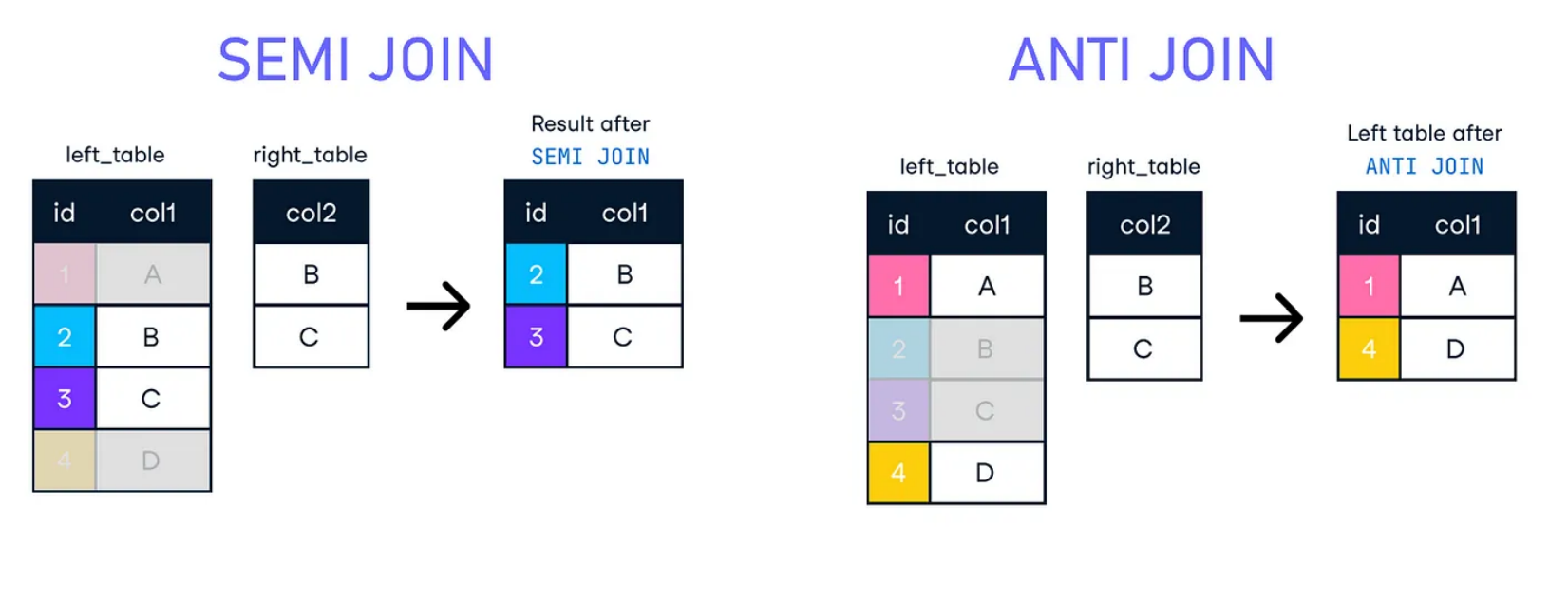


Figure 12. Summary of SEMI JOIN and ANTI JOIN from DataCamp SQL JOINS Cheat Sheet

. . .

So far, we've already learned how to use `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `SELF JOIN`, and `CROSS JOIN` to combine multiple tables based on a related column between them. We've learned set operations such as `UNION`, `UNION ALL`, `INTERSECT`, `EXECPT` and how to use them. And last but not least we've learned simple subquery in `SEMI JOIN` and `ANTI JOIN`. See you in the next section of a beginner guide for learning SQL from scratch.

. . .

This is part of the Intermediate SQL series, here is the list of Intermediate SQL series.

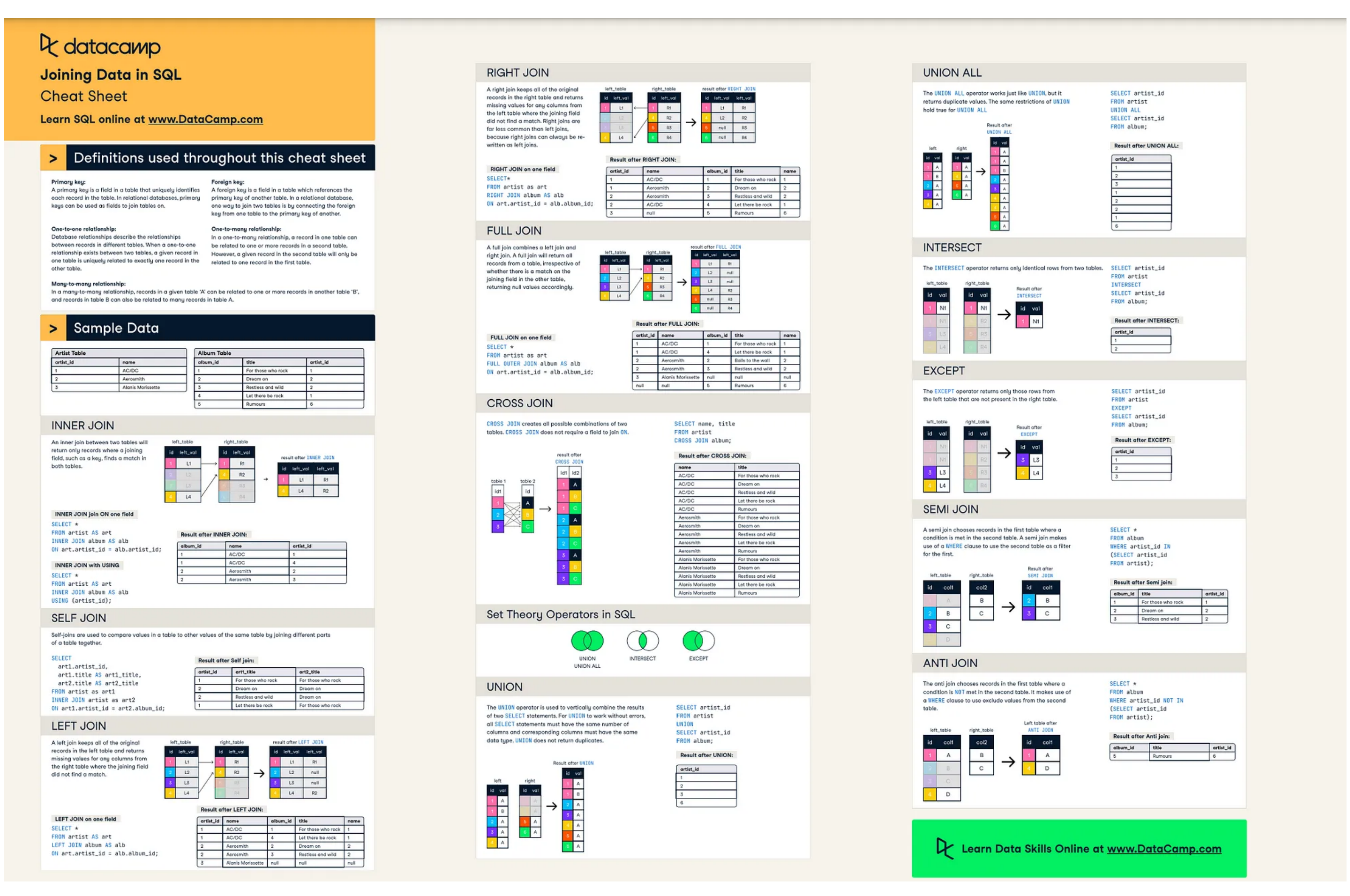
Intermediate SQL (Part 2)
A beginner guide for learning SQL from scratch
medium.com

. . .

Reference

1. SQL Joins Cheat Sheet

https://www.datacamp.com/cheat-sheet/sql-joins-cheat-sheet


datacamp
Joining Data in SQL
Cheat Sheet
Definitions used throughout this cheat sheet
Sample Data
INNER JOIN
SELF JOIN
LEFT JOIN
RIGHT JOIN
FULL JOIN
CROSS JOIN
Set Theory Operators in SQL
UNION
INTERSECT
EXCEPT
UNION ALL
SEMI JOIN
ANTI JOIN
Learn Data Skills Online at www.DataCamp.com

SQL Joins Cheat Sheet

2. Inner Join in SQL Bolt

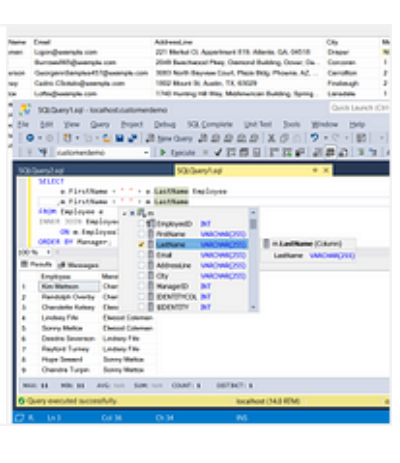
Multi-table queries with JOINs
SQLBolt provides a set of interactive lessons and exercises to help you learn SQL
sqlbolt.com

3. Outer Joins in SQL Bolt

SQLBolt - Learn SQL - SQL Lesson 7: OUTER JOINs
SQLBolt provides a set of interactive lessons and exercises to help you learn SQL
sqlbolt.com

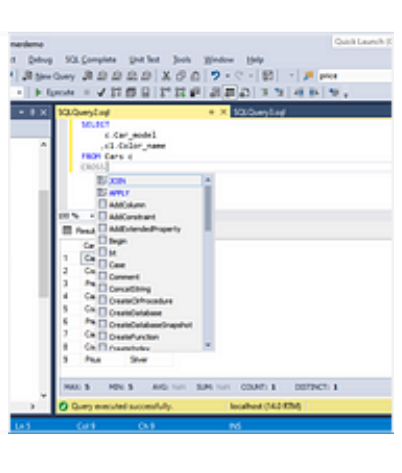
4. Understanding SQL Server SELF JOIN By Practical Examples

dbForge SQL Complete - Powerful T-SQL Formatting Tool
Learn about SELF JOIN in SQL with simple examples: SQL SELF JOIN tutorial on the use of SELF JOINs.
www.devart.com



5. Understanding SQL CROSS JOIN

dbForge SQL Complete - Powerful T-SQL Formatting Tool
Learn about SQL CROSS JOIN and how to use it to create a Cartesian product of the joined tables.
www.devart.com



6. Set Operations in SQL

Set operations (SQL) - Wikipedia
From Wikipedia, the free encyclopedia Set operations in SQL is a type of operations which allow the results of multiple…
en.wikipedia.org

7. Set Operations in MySQL

MySQL :: MySQL 8.0 Reference Manual :: 13.2.14 Set Operations with UNION, INTERSECT, and EXCEPT
SQL set operations combine the results of multiple query blocks into a single result. A query block, sometimes also…
dev.mysql.com

8. Combine tables with UNION, INTERSECT, and EXCEPT

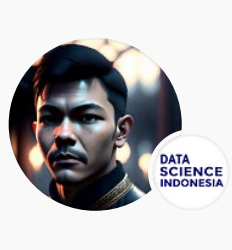
Combine tables with UNION, INTERSECT, and EXCEPT
With the standard set operations UNION, INTERSECT, and EXCEPT/MINUS, you can combine the results from two or more…
www.ibm.com

9. MySQL UNION Explained

MySQL UNION Explained: A Tutorial with Practical Examples for All Skill Levels - Devart Blog
Explore MySQL UNION in-depth with our step-by-step tutorial. Get insights into merging query results and optimizing…
blog.devart.com

MySQL Postgresql Data Analysis Sql

24



Written by Faisal Afif

Follow

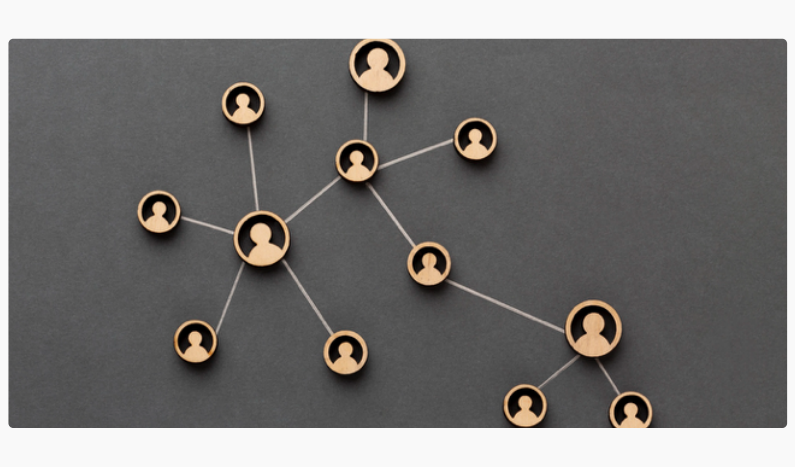
44 Followers · Writer for Data Science Indonesia

A lifelong learner sharing about data analytics and data science | Medium medium.com/@faisal.afif | LinkedIn linkedin.com/in/faisal-afif/

More from Faisal Afif and Data Science Indonesia

Faisal Afif

Logistics and Supply Chain Analytics by Utilizing Tableau…
This article is part of Business Analytics course final project—Pacmann.
16 min read · Nov 5, 2023
69

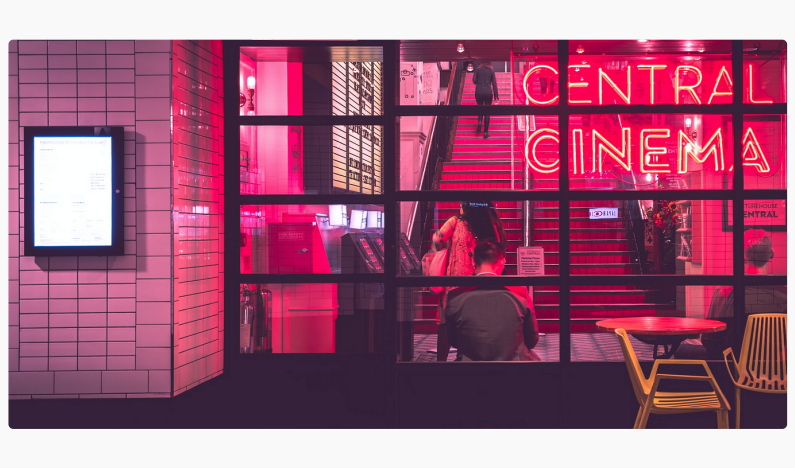


Dede Kurniawan in Data Science Indonesia

Python 02: Array pada Python (List, Tuple, Dictionary, dan Set)
Memperkenalkan beberapa jenis tipe data di Python yang merepresentasikan data…
14 min read · Oct 21, 2022
51

Dede Kurniawan in Data Science Indonesia

Statistika Deskriptif: Pengukuran Central Tendency, Variation, dan…
Berisi penjelasan dan implementasi perhitungan menggunakan Python dan…
8 min read · Aug 28, 2022
19



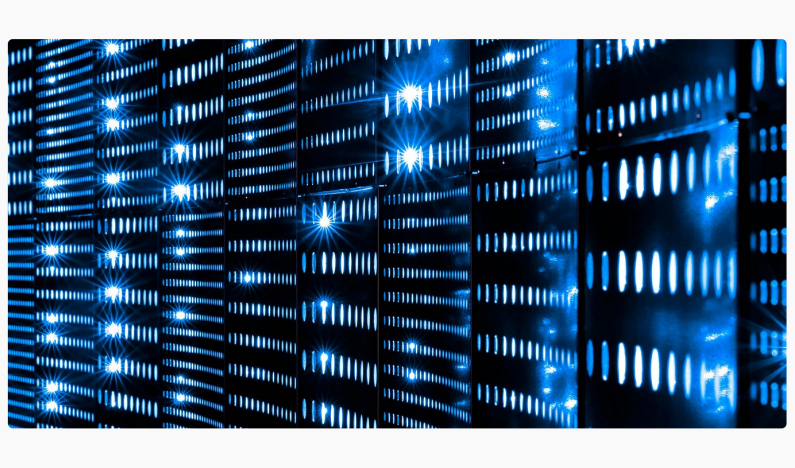
Faisal Afif in Data Science Indonesia

Introduction to SQL (Part 2)
A beginner guide for learning SQL from scratch
6 min read · Oct 8, 2023
1

See all from Faisal Afif

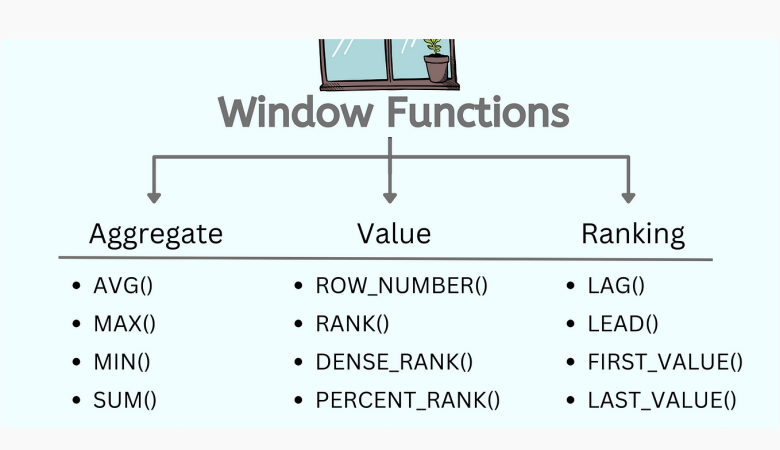
See all from Data Science Indonesia

Recommended from Medium



Data SQL

SQL Optimization and Performance Tuning
Here, We will discuss for SQL optimization and performance tuning.
2 min read · Mar 19, 2024



Hasan Hüseyin Coşgun

Learn SQL Window Functions With Queries
Time to discover the world of Window Functions with examples; lead( ), lag( ),…
7 min read · Nov 2, 2023
88

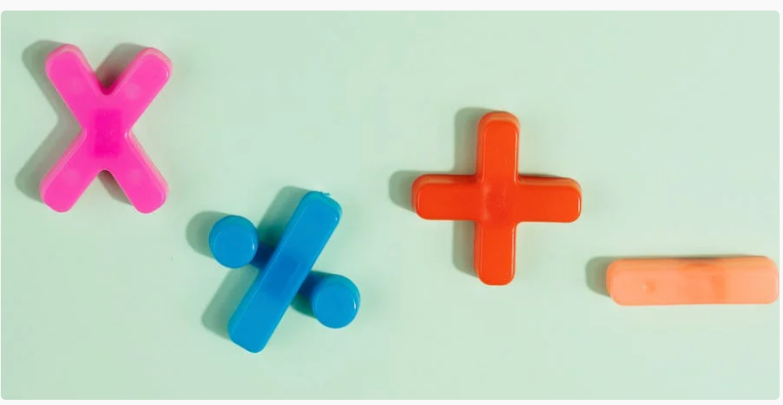
Lists

Practical Guides to Machine Learning
10 stories · 1333 saves

ChatGPT prompts
47 stories · 1447 saves

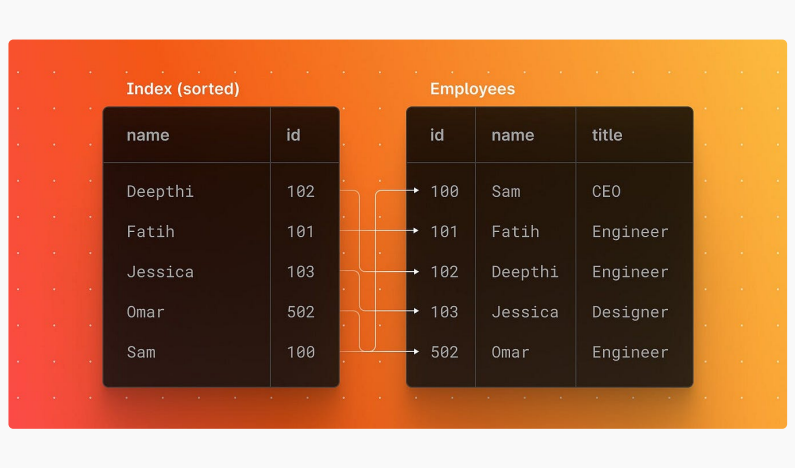
ChatGPT
21 stories · 584 saves

Natural Language Processing
1383 stories · 878 saves



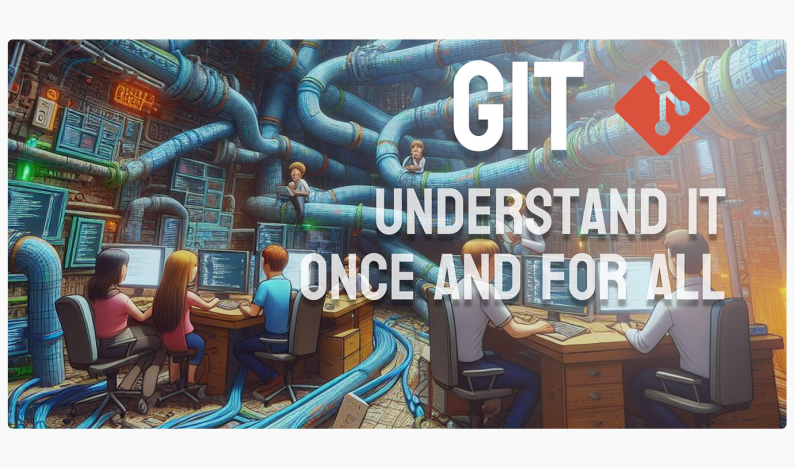
SQL Programming

Mastering SQL Aggregation Functions: A Comprehensive Gui…
Aggregation functions play an important role in SQL when it comes to summarizing and…
5 min read · Dec 29, 2023



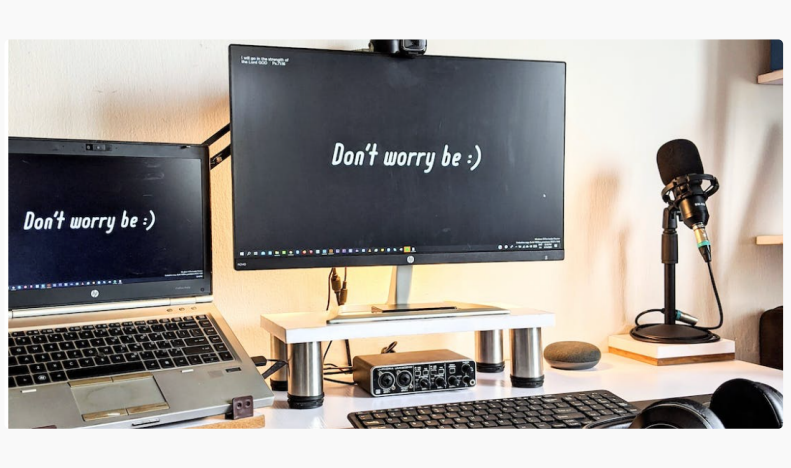
Ekrem Sönmezer in Picus Security Engineering

Deep Dive Into SQL Indexes
An index is a type of data structure in databases which aims to improve speed and…
9 min read · Dec 4, 2023
48



Jake Page

The guide to Git I never had.
Doctors have stethoscopes.
13 min read · Apr 11, 2024
2K 20



Gözde Madendere Barın

Exploring SQL Subqueries
SQL serves as the backbone for managing and manipulating data. While SQL provides…
3 min read · 6 days ago
90

See more recommendations